IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

(A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY))

BALMATTA, MANGALORE

HIGH LEVEL DESIGN
ON
SUPPLY CHAIN MANAGEMENT SYSTEM

SUBMITTED BY

ABHAY V

REGNO :

23BCAICD004

LH-05

GUIDED BY

MS. YUKTHA

SUBMISSION DATE

24-04-2026

Table of Contents

1. Introduction
   1.1 Scope
   1.2 Intended Audience
   1.3 System Overview
2. System Design
   2.1 Application Design
   2.2 Process Flow
   2.3 Information Flow
   2.4 Component Design
   2.5 Key Design Considerations
   2.6 API Catalogue
3. Data Design
   3.1 Data Model
   3.2 Data Access Mechanism
   3.3 Data Retention Policies
   3.4 Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   7.1 Security Aspects
   7.2 Performance Aspects
8. Conclusion
9. References

1. Introduction

1.1 Scope

This document defines the architecture, modules, workflows, and system behavior of the Supply Chain Management System in detail. It covers the complete design of all system components including inventory management, shipment tracking, API endpoint design, and data management strategies adopted for this version.

The Supply Chain Management System is built to handle end-to-end operations of inventory control and shipment lifecycle management. This document serves as the primary technical reference for all design decisions made during development.

1.2 Intended Audience

This document is intended for the following stakeholders:

- Developers responsible for implementing the system modules and API endpoints.
- Testers who will design and execute test cases based on the described system behavior.
- Project evaluators and academic reviewers assessing the overall design quality.
- Stakeholders interested in understanding the functional and non-functional characteristics of the system.

1.3 System Overview

The Supply Chain Management System is a FastAPI-based backend application designed to manage inventory and shipment operations efficiently. The system provides RESTful API endpoints that allow clients to perform CRUD (Create, Read, Update, Delete) operations on inventory items and manage the complete lifecycle of shipments from creation to delivery.

The system is designed with simplicity and performance in mind, leveraging FastAPI's asynchronous capabilities for low-latency responses. In-memory data storage is used in the current version, which is appropriate for demonstration and testing environments. The architecture is modular, allowing each component to be developed, tested, and replaced independently without affecting other parts of the system.

Key capabilities of the system include real-time inventory tracking, shipment status management, input data validation, and consistently structured API responses. The system is fully stateless — each HTTP request is processed independently without relying on any server-side session data.

2. System Design

2.1 Application Design

The system follows a three-layer architecture consisting of the Presentation Layer (API), the Business Logic Layer, and the Data Layer. This clear separation of concerns ensures that each layer is independently maintainable, testable, and replaceable.

- Presentation Layer (API): Exposes RESTful endpoints via FastAPI. Handles HTTP request parsing, input deserialization using Pydantic models, and response serialization.

- Business Logic Layer: Contains core processing logic for inventory and shipment operations. Validates business rules, orchestrates data interactions, and generates appropriate structured responses.
- Data Layer: Manages in-memory storage using Python lists. All CRUD operations are abstracted through dedicated helper functions invoked by the business logic layer.

2.2 Process Flow

The following represents the step-by-step execution flow when a client makes an API request:

```
[User Request]
       ↓
[API Endpoint]          ←  FastAPI route handler
       ↓
[Validation Layer]      ←  Pydantic model validation
       ↓
[Business Logic]        ←  Core processing functions
       ↓
[Database / Memory]     ←  In-memory Python lists
       ↓
[Response Returned]     ←  JSON formatted response
```

If validation fails at the Validation Layer, an HTTP 422 Unprocessable Entity response is immediately returned without further processing. Successful operations return HTTP 200 (OK) or HTTP 201 (Created) with structured JSON payloads.

2.3 Information Flow

Information flows unidirectionally from the client through each system layer and back. Requests carry JSON payloads deserialized into Pydantic models at the API layer, passed to business logic, and then forwarded to the data layer. Responses are serialized back to JSON by FastAPI.

```
Client  ►  API Layer  ►  Business Logic  ►  Data Layer
Client  ◄  API Layer  ◄  Business Logic  ◄  Data Layer
```

2.4 Component Design

The system is composed of two primary functional modules — Inventory and Shipment — sharing a common Core Logic layer:

```
[Inventory Module]           [Shipment Module]
        ↓                            ↓
        ---------->[Core Logic]<-----------
                        ↓
                [Response Layer]
```

- Inventory Module: Handles all operations related to inventory items. Maintains inventory list and enforces unique item IDs.

- Shipment Module: Manages creation and tracking of shipments. Each shipment references an inventory item and tracks destination, quantity, and status.
- Core Logic: Shared helper functions for searching, filtering, and updating in-memory records — used by both modules to avoid duplication.
- Response Layer: Formats business logic output into consistent JSON responses with appropriate HTTP status codes.

2.5 Key Design Considerations

- Performance: FastAPI's async handling ensures minimal latency; in-memory storage avoids database overhead.
- Modularity: Inventory and Shipment modules are fully separated with clean interfaces, enabling independent replacement or extension.
- Scalability: Stateless design allows multiple instances to run concurrently behind a load balancer without sticky sessions.
- Maintainability: Clear separation of concerns and consistent coding patterns reduce complexity and ease future development.

2.6 API Catalogue

The following REST API endpoints are exposed by the system:

- GET /inventory — Retrieve a list of all inventory items.
- POST /inventory — Create a new inventory item with provided details.
- PUT /inventory/{id} — Update an existing inventory item identified by its ID.
- DELETE /inventory/{id} — Permanently remove an inventory item by its ID.
- POST /shipments — Create a new shipment record linked to an inventory item.
- GET /shipments — Retrieve all shipment records currently in the system.
- PUT /shipments/{id} — Update the status or details of an existing shipment by its ID.

3. Data Design

3.1 Data Model

The system uses two primary data entities stored entirely in-memory:

Inventory Entity:

- id (integer) — Unique identifier for the inventory item.
- name (string) — Descriptive name of the inventory item.
- quantity (integer) — Current stock quantity available in the warehouse.
- location (string) — Storage location or warehouse identifier for the item.

Shipment Entity:

- id (integer) — Unique identifier for the shipment record.
- item_id (integer) — Reference to the associated inventory item being shipped.
- destination (string) — Target delivery address or location code for the shipment.
- quantity (integer) — Number of inventory units included in this shipment.

- status (string) — Current lifecycle status: Pending, In Transit, or Delivered.

3.2 Data Access Mechanism

All data is stored in Python lists maintained in application memory. Access is provided through a set of dedicated helper functions that abstract the storage mechanism from the business logic layer. This abstraction ensures that if the storage backend is changed in the future (e.g., replaced with a database), only the data access functions need to be updated.

Read operations iterate through the in-memory list to locate records matching a given ID or filter criteria. Write operations append new records or modify existing entries in-place using indexed list access. Delete operations rebuild the list by filtering out records matching the target ID.

3.3 Data Retention Policies

In the current version (v1.0), all data is held exclusively in application memory. All inventory and shipment records are permanently lost when the application process is terminated or restarted. There is no persistence layer, no file-based storage, and no database connection in this version.

This design is intentional within the scope of this version, which focuses on demonstrating the API structure, request-response flows, and business logic. Persistence will be introduced in a future release using a relational or NoSQL database backend.

3.4 Data Migration

No data migration strategy is required in the current version since no persistent database is used. All data structures are initialized fresh each time the application starts.

When a persistent database is introduced in future versions, a formal migration plan using tools such as Alembic for SQLAlchemy-based relational databases will be implemented. This will allow schema changes and data transitions between application versions to be managed in a structured and reversible manner.

4. Interfaces

The Supply Chain Management System exposes its functionality exclusively through REST API interfaces. There are no graphical user interfaces (GUI) or command-line interfaces (CLI) in the current version.

External Interface: Clients interact with the system by sending HTTP requests to the defined API endpoints over standard HTTP/HTTPS. All request bodies must be formatted as JSON. All API responses are returned as JSON with appropriate HTTP status codes indicating success or failure.

Internal Interface: Internally, the system uses direct Python function calls between the API layer and the business logic layer. Business logic functions call data access helpers in the data layer. All internal communication is synchronous within a single application process.

Error Interface: Validation errors return HTTP 422 Unprocessable Entity responses with field-level error details generated by Pydantic. Record not found errors return HTTP 404 Not Found. Unexpected server errors return HTTP 500 Internal Server Error with a generic message to prevent internal detail exposure.

5. State and Session Management

The Supply Chain Management System is designed to be fully stateless in accordance with REST architectural principles. Each HTTP request received by the API is processed independently, with no dependency on any prior request or stored session context.

There is no server-side session storage, no cookies, and no authentication tokens in the current version. Each request must contain all information necessary for the server to process it completely and return a response. This eliminates the need for session affinity or sticky routing in load-balanced environments.

The stateless architecture improves system reliability and simplifies horizontal scaling. If a server instance fails, requests can be routed to any other instance without requiring session state recovery. This design is consistent with RESTful API best practices and supports future deployment in containerized or cloud-native environments.

6. Caching

No caching mechanism is implemented in the current version of the Supply Chain Management System. All API requests result in direct reads from or writes to the in-memory data store, with no intermediate cache layer.

Since the data store is already in-memory, read operations are inherently fast, making an additional cache layer unnecessary for this version. Response times remain consistently low without caching overhead.

When a persistent database is introduced in future versions, a caching layer such as Redis will be evaluated for use with frequently accessed, read-heavy endpoints such as GET /inventory. Cache invalidation strategies such as time-based TTL expiry and write-through invalidation will be implemented to maintain data consistency between the cache and the underlying database.

7. Non-Functional Requirements

7.1 Security Aspects

The current version implements foundational security measures appropriate for a demonstration and evaluation context:

- Input Validation: All incoming request data is validated using Pydantic models before reaching the business logic layer. Invalid data types, missing required fields, and constraint violations are rejected with structured error messages before any processing occurs.

- Injection Prevention: By using Pydantic for data parsing and Python in-memory lists for storage, the system avoids SQL injection and similar injection vulnerabilities that affect database-backed applications.
- Error Information Control: Internal error stack traces and implementation details are never exposed in API responses. Generic messages are returned for unexpected failures to prevent information leakage to potential attackers.
- Data Integrity: Helper functions enforce referential checks — for example, shipment creation validates that the referenced inventory item exists before proceeding.

Planned security enhancements for future versions include API key or JWT-based authentication, role-based access control (RBAC), HTTPS enforcement via TLS, and rate limiting to prevent abuse and denial-of-service scenarios.

7.2 Performance Aspects

FastAPI is built on Starlette and Python's asyncio framework, enabling asynchronous handling of concurrent requests without blocking the main thread. This architecture delivers high throughput and consistent low latency across all API endpoints.

Key performance characteristics of the current system:

- Response Latency: Typical API responses are expected under 50ms for all standard CRUD operations due to in-memory data access and FastAPI's minimal processing overhead.
- Throughput: FastAPI benchmarks place it among the fastest Python web frameworks, capable of handling thousands of requests per second on standard commodity hardware.
- Memory Efficiency: In-memory storage consumes memory proportional to the number of active records. Within the expected data volumes for this version, memory usage remains well within normal operating limits.
- Startup Time: The application initializes in under two seconds, making it suitable for containerized environments and serverless platforms where fast cold starts are essential.
- Concurrency: Python's asyncio enables non-blocking I/O, allowing the system to handle multiple simultaneous client connections efficiently without spawning additional threads or processes.

8. Conclusion

The Supply Chain Management System provides a well-structured, efficient, and maintainable solution for managing inventory and shipment operations through a clearly defined RESTful API interface. The system successfully enables full CRUD management of inventory items and comprehensive lifecycle management of shipments, delivered with high performance through the FastAPI framework.

The three-layer architecture establishes a clean separation of concerns between the API, business logic, and data layers. This modular structure ensures each component can be developed, tested, and modified independently, reducing the cost and risk of future enhancements.

The stateless design aligns with RESTful principles and positions the system for scalable deployment. The data access layer is abstracted behind helper functions, allowing a persistent database to be introduced in future releases with minimal disruption to existing layers.

In summary, the Supply Chain Management System demonstrates a sound approach to API-driven backend design. It is modular, scalable in architecture, and well-positioned for growth into a production-ready supply chain platform in subsequent phases.

9. References

- FastAPI Official Documentation — https://fastapi.tiangolo.com
- Python Official Documentation — https://docs.python.org/3/
- Pydantic Documentation — https://docs.pydantic.dev
- Starlette Framework — https://www.starlette.io
- Uvicorn ASGI Server — https://www.uvicorn.org
- Architectural Styles and the Design of Network-based Software Architectures — Roy Fielding, 2000.
- RESTful API Design Best Practices — Apigee.
- Project Source Code — Internal repository.